Experiment 2: Production Rules and the Match Phase

Aim

To represent domain knowledge as IF–THEN production rules, hold facts in a working memory, and compute the conflict set of applicable rules.

Theory

A production system has three parts: a rule base of IF–THEN productions, a working memory (WM) of currently believed facts, and an inference engine that runs the recognise–act cycle: match all rules against WM, select one rule from the resulting conflict set, act by executing its conclusion, and repeat.

This experiment isolates the match phase. A rule is applicable when every condition in its IF part is present in WM and its conclusion is not already there (refractoriness — preventing a rule from firing repeatedly on the same data). The set of applicable rules is the conflict set; choosing among them is the subject of Experiment 5.

Algorithm

1. Encode each rule as a record with an identifier, a list of conditions and a conclusion.
2. Initialise working memory as a set of facts.
3. For each rule, test whether all its conditions are members of WM and its conclusion is absent.
4. Collect and display the conflict set.

Procedure

1. Define the knowledge using **IF–THEN production rules**.
2. Store the current facts in **working memory**.
3. Compare the conditions of each rule with the facts in working memory.
4. If all the conditions of a rule are satisfied, consider that rule as a matching rule.
5. Collect all matching rules into the **conflict set**.
6. Run the program with the given working-memory facts.
7. Modify the facts and observe how the conflict set changes

Observations

The conflict set changes when new facts are added to working memory. When all the conditions of a rule are satisfied, that rule becomes applicable and is added to the conflict set. If any condition is missing, the rule does not match. This shows how the match phase identifies the rules that can be applied to the current facts.

Exercises

1. Add facts to WM one at a time and record how the conflict set grows or shrinks.
2. Extend the matcher to support negated conditions of the form ‘NOT fact’.

3. Measure (with time.perf_counter) how match time grows as you scale to 1,000 auto-generated rules, and relate the result to why the RETE algorithm exists.